

GET STARTED

[Welcome to SP-API Documentation](#)[What is the Selling Partner API?](#)[Selling Partner API Onboarding Overview](#)[Terminology](#)[Frequently Asked Questions](#)

CHANGELOG

[SP-API Release Notes](#)[SP-API Deprecation Schedule](#)[SP-API Product Metadata Updates](#)

DIRECTORIES

[SP-API Models](#)[SP-API Seller Use Cases](#)

Javascript Example of Meta-Schema v1

Validate payloads with Amazon Product Type Definitions meta-schema instances.

Example Validator Implementation for Javascript

For JavaScript applications, the [Hyperjump - JSON Schema Validator](#) library supports [JSON Schema Draft 2019-09](#) and custom vocabularies. The following example uses Node.js to demonstrate how to utilize the [Hyperjump - JSON Schema Validator](#) library to validate payloads with instances of the *Amazon Product Type Definition Meta-Schema*. The following example code will not work in a web browser, however, the library does not have dependencies on Node.js and can be used in a web browser. There is no requirement to use this specific library or the example implementation. Amazon does not provide technical support for third-party JSON Schema libraries and this is provided as an example only.

Schema Configuration

When using the [Hyperjump - JSON Schema Validator](#) to validate instances of the *Amazon Product Type Definition Meta Schema* with custom vocabulary, the meta-schema is configured as a `JsonSchema` instance with custom vocabulary keyword validation registered in the [Hyperjump - JSON Schema Core](#).

Constants:

JavaScript

```
// $id of the Amazon Product Type Definition Meta-Schema.
const schemaId = "https://schemas.amazon.com/selling-partners/definitions/product-types/meta-schema/v1";
// $id of the Amazon Product Type Definition Luggage-Schema.
const luggageSchemaId = "https://schemas.amazon.com/selling-partners/definitions/product-types/schema/v1/LUGGAGE";

// Local copy of the Amazon Product Type Definition Meta-Schema.
const metaSchemaPath = "./amazon-product-type-definition-meta-schema-v1.json";

// Local copy of an instance of the Amazon Product Type Definition Meta-Schema.
const luggageSchemaPath = "./luggage.json";
```

Configure Meta-Schema:

JavaScript

```
// Add custom vocabulary and keywords
const { Core, Schema } = require("@hyperjump/json-schema-core");
const maxUniqueItemsKeyword =
  require("./keywords/MaxUniqueItemsKeyword.js");
const maxUtf8ByteLengthKeyword =
  require("./keywords/MaxUtf8ByteLengthKeyword.js");
const minUtf8ByteLengthKeyword =
```

SP-API Vendor Use Cases

Seller Central URLs

Vendor Central URLs

REGISTRATION

SP-API Registration Overview >

Developer Registration Request Status

Third-Party Provider Registration >

Service Provider Registration and Role Approval Process Overview

Website Guidelines for Public Developers and Service Providers

User Permissions for Service Providers

Register your Application

Update your Application Information

Rotate your Application's LWA Credentials >

View your Application Information and Credentials

Learn How Sellers Authorize Service Providers

Selling Partner API Roles >

Policies and Agreements

About Facial Data

AUTHORIZATION

Authorize Applications >

Renew Authorizations

Revoke Authorizations

Authorization Limits

Special Authorization Types >

INTEGRATION

Building Robust Amazon SP-API Applications

SP-API SDKs >

Javascript Example of Meta-Schema v1

```
require("../keywords/MinUtf8ByteLengthKeyword.js");
const customVocabularyId = "https://schemas.amazon.com/selling-
partners/definitions/product-types/vocabulary/v1";

// Configure $vocabulary defined in the schema JSON as vocabularyToken
Schema.setConfig(schemaId, "vocabularyToken", "$vocabulary");

// Add custom vocabulary and keywords
Core.defineVocabulary(customVocabularyId, {
  maxUniqueItems : maxUniqueItemsKeyword,
  minUtf8ByteLength : minUtf8ByteLengthKeyword,
  maxUtf8ByteLength : maxUtf8ByteLengthKeyword
});

// Add local schema JSON in JsonSchema.add() to use local copies of schemas
rather than retrieving them from the web.
const JsonSchema = require("@hyperjump/json-schema");

// Add meta schema
const metaSchemaJSON = JSON.parse(fs.readFileSync(metaSchemaPath), "utf8");
JsonSchema.add(metaSchemaJSON, schemaId);

// Add luggage schema
const luggageSchemaJSON = JSON.parse(fs.readFileSync(luggageSchemaPath),
"utf8");
JsonSchema.add(luggageSchemaJSON, luggageSchemaId, schemaId);
```

Load Meta-Schema Instance:

JavaScript

```
// Retrieve previously added luggage schema from JsonSchema instance
var luggageSchema = await JsonSchema.get(luggageSchemaId);
```

Payload Validation

With an instance of the *Amazon Product Type Definition Meta-Schema* loaded as a `JsonSchema` instance, payloads can be validated using the instance.

JavaScript

```
// Load JSON for the payload (this can be constructed in code or read from
a file).
const payload = JSON.parse(fs.readFileSync("../payload.json"), "utf8");

// Validate the payload and get validation result.
const result = await JsonSchema.validate(luggageSchema, payload,
JsonSchema.BASIC);
const isValid = result.valid;
```

If the payload is valid, `isValid` will return `true` with an empty list of error keywords. Otherwise `isValid` will return `false` with a list of error keywords.

Example Validation Message:

JavaScript

```
{
  keyword: 'https://schemas.amazon.com/selling-
partners/definitions/product-types/meta-schema/v1#minUtf8ByteLength',
  absoluteKeywordLocation: 'https://schemas.amazon.com/selling-
partners/definitions/product-
types/schema/v1/LUGGAGE#/properties/contribution_sku/items/properties/value/min
instanceLocation: '#/contribution_sku/0/value',
  valid: false
}
```

Keyword Validation

The [Hyperjump - JSON Schema Core](#) provides a framework that can be used to validate custom vocabulary, keywords, and other tools.

The following examples illustrate implementations of the `JsonSchema` that validate the custom vocabulary in instances of the *Amazon Product Type Definition Meta-Schema*.

MaxUniqueItemsKeyword script

JavaScript

```
const { Schema, Instance } = require("@hyperjump/json-schema-core");

const compile = (schema, ast, parentSchema) => {
  const maxUniqueItems = Schema.value(schema);
  // Retrieve the selectors defined in meta-schema
  const selectors = parentSchema.value.selectors;
  return [maxUniqueItems, selectors];
}

const interpret = ([maxUniqueItems, selectors], instance, ast) => {
  if (!Instance.typeOf(instance, "array")) {
    return false;
  }

  let uniqueItems = new Array();
  // For each instance in the example schema retrieve selectors
  combinations
  instance.value.forEach(inst => {
    let selectorCombination = {};
    Object.entries(inst).forEach(([key, value]) => {
      if(selectors !== undefined && selectors.includes(key)) {
        selectorCombination[key] = value;
      }
    });
    uniqueItems.push(JSON.stringify(selectorCombination));
  });

  let countMap = new Map();
  // Add count of each unique selector combination in countMap
  uniqueItems.forEach(item => {
    countMap.get(item) !== undefined ? countMap.set(item,
countMap.get(item)+ 1): countMap.set(item, 1);
  });
  // Filter the countMap for values greater than maxUniqueItems
  let filteredItems = Array.from(countMap.entries()).filter(item => {
    return item[1] > maxUniqueItems;
  });
  // If filteredItems found then validation fails, else succeeds
  return filteredItems.length > 0 ? false : true;
};

module.exports = { compile, interpret };
```

MaxUtf8ByteLengthKeyword script

JavaScript

```
const { Schema, Instance } = require("@hyperjump/json-schema-core");

const compile = (schema, ast) => Schema.value(schema);

const interpret = (maxUtf8ByteLength, instance, ast) =>
Instance.typeOf(instance, "string") &&
  Buffer.byteLength(Instance.value(instance)) <= maxUtf8ByteLength;

module.exports = { compile, interpret };
```

Delete an Application From Your Developer Account

CALLING THE SP-API

Configuration Details

Call Structure

Development Tools

Performance Management

SOLUTION PROVIDER PORTAL

Manage Users in Solution Provider Portal

Manage Your Business And Contact information in Solution Provider Portal

Update Your Developer Profile in Solution Provider Portal

Unify Your Developer Accounts

Verify Your Identity in Solution Provider Portal

API Usage Metrics in Solution Provider Portal

TUTORIALS

Tutorial: Subscribe to the ORDER_CHANGE Notification

Tutorial: Test Selling Partner API Endpoints

Tutorial: Automate your SP-API Calls Using a C# SDK

MinUtf8ByteLengthKeyword script

```
JavaScript
const { Schema, Instance } = require("@hyperjump/json-schema-core");

const compile = (schema, ast) => Schema.value(schema);

const interpret = (minUtf8ByteLength, instance, ast) =>
  Instance.typeOf(instance, "string") &&
  Buffer.byteLength(Instance.value(instance)) >= minUtf8ByteLength;

module.exports = { compile, interpret };
```

Updated 19 days ago

← C# Example of Meta-Schema v1 Java Example of Meta-Schema v1 →

Did this page help you? ☐ Yes ☐ No

TROUBLESHOOTING

Policies and Agreements

Troubleshoot SP-API Errors

URL Encoding

Resolve Common HTTP and Authorization Error Codes

External Fulfillment Errors

Listings Items API Issues Troubleshooting

SELLING PARTNER APPSTORE

What is the Selling Partner Appstore?

List Your App on the Selling Partner Appstore

Edit Your Appstore Listing

Check Listing Status

Appstore Ratings and Reviews

Press Releases and Promotions

SERVICE PROVIDER NETWORK

List Your Service

SECURITY AND COMPLIANCE

SP-API Security and Compliance Overview

Amazon Selling Partner API Guard Implementation Guide

Privacy notice | Conditions of use

VAT Calculation Service >

Amazon Seller Data Access
Best Practices >

A+ CONTENT API
A+ Content API >

A+ Content Examples

AMAZON WAREHOUSING AND DISTRIBUTION API
Amazon Warehousing and Distribution API >

Amazon Warehousing and Distribution API v2024-05-09
Reference

APP INTEGRATIONS API
App Integrations API ∨

APPLICATION MANAGEMENT API

Application Management API >

CATALOG ITEMS API

Catalog Items API >

Catalog Items API Rate Limits
Catalog Items API v2022-04-01 Reference

CUSTOMER FEEDBACK API

Customer Feedback API >

Customer Feedback API Rate Limits

DATA KIOSK

Data Kiosk API >

Data Kiosk Schema Explorer User Guide
Data Kiosk Query Processing Finished Notification
Building Data Kiosk workflows guide
Vendor Analytics Dataset Use Case Guide
Schedule Data Kiosk Queries

DELIVERY BY AMAZON API

Delivery by Amazon API >

EASY SHIP API

Easy Ship API >

EXTERNAL FULFILLMENT

External Fulfillment Inventory API >

External Fulfillment Returns API	>
External Fulfillment Shipping API	>
External Fulfillment Errors	
External Fulfillment APIs Rate Limits	
FULFILLMENT BY AMAZON (FBA)	
FBA Inventory API	>
FBA Inventory Dynamic Sandbox Guide	
FEEDS API	
Feeds API	>
Feed Type Values	>
Feeds JSON Schemas	↗
Feeds API Best Practices	
Feeds API FAQ	
Feeds API Rate Limits	
FINANCES	
Finances API	>
Transfers API	>
FULFILLMENT INBOUND API	
Fulfillment Inbound API	>

Fulfillment Inbound API FAQ

Migrating Fulfillment Inbound workflows

Fulfillment Inbound API Rate Limits

FULFILLMENT OUTBOUND API

Fulfillment Outbound Dynamic Sandbox Guide

Fulfillment Outbound API >

Fulfillment Outbound API Rate Limits

MCF Best Practices

INVOICING

Invoices API >

Invoices API FAQ

LISTINGS

Listings Items API >

Listings Items API Rate Limits

Listings Restrictions API >

Listings Restrictions API Rate Limits

Manage Product Listings with the Selling Partner API

Building Listings Management Workflows Guide

Understanding Amazon listing status and seller-fulfilled inventory management

Listings Management Workflow Migration >

Manage Amazon Haul, Advanced Multiple-Offer, and Multiple-Fulfillment Use Cases

Listings APIs FAQ

Product Type Definitions API >

Search available Product Type Definitions

Get Product Type Definition recommendations

Get recommended browse nodes or item type keywords

Retrieve a Product Type Definition

Amazon Product Type Definitions Meta-Schema (v1)

C# Example of Meta-Schema v1

Javascript Example of Meta-Schema v1

Java Example of Meta-Schema v1

Product Type Definitions API Rate Limits

MERCHANT FULFILLMENT API

Merchant Fulfillment API >

MESSAGING API

Messaging API >

NOTIFICATIONS API

Notifications API >

Notification Type Values

Tutorial: Subscribe to the ORDER_CHANGE Notification ↗

ORDERS API

Orders API >

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier ↗

Orders API Migration Guide

Orders API Rate Limits

PRODUCT FEES API

Product Fees API >

PRODUCT PRICING API

Product Pricing API >

Product Pricing API and Notifications FAQ

Price Adjustment Automation Workflows Guide

Manage automated pricing rules with SP-API

REPLENISHMENT API

Replenishment API >

REPORTS API

Reports API >

Report Type Values >

Reports JSON Schemas ↗

Reports API FAQ

SALES API

Sales API >

SELLER WALLET API

Seller Wallet API >

Seller Wallet API Rate Limits

SELLERS API

Sellers API >

SERVICES API

Services API >

SHIPMENT INVOICING API

Shipment Invoicing API >

SHIPPING API

Shipping API v2 Resources ↗

Shipping API v1 Reference

SOLICITATIONS API

Solicitations API >

SUPPLY SOURCES API

Supply Sources API >

Multi-Location Inventory Integration Guide

Supply Sources API Rate Limits

TOKENS API

Tokens API >

UPLOADS API

Uploads API >

VEHICLES API

Vehicles API >

Vehicles API Rate Limits

VENDOR DIRECT FULFILLMENT APIS

Vendor Direct Fulfillment Dynamic Sandbox Guide

Vendor Direct Fulfillment Workflow Guide

SP-API Bill-to-Party Addresses

VENDOR DIRECT FULFILLMENT TRANSACTION STATUS API

Vendor Direct Fulfillment Transaction Status API >

VENDOR DIRECT FULFILLMENT INVENTORY API

Vendor Direct Fulfillment Inventory API >

VENDOR DIRECT FULFILLMENT ORDERS API

Vendor Direct Fulfillment Orders API >

VENDOR DIRECT FULFILLMENT SHIPPING API

Vendor Direct Fulfillment Shipping API >

VENDOR DIRECT FULFILLMENT PAYMENTS API

Vendor Direct Fulfillment Payments API >

VENDOR RETAIL PROCUREMENT INVOICES API

Vendor Retail Procurement Invoices API >

VENDOR RETAIL PROCUREMENT ORDERS API

Vendor Retail Procurement Orders API >

VENDOR RETAIL PROCUREMENT SHIPMENTS API

Vendor Retail Procurement Shipments API >

**VENDOR RETAIL PROCUREMENT TRANSACTION STATUS
API**

Vendor Retail Procurement Transaction Status API >

